

计算机视觉 课程实验报告

学号：202300130183	姓名：宋浩宇	
-----------------	--------	--

实验题目：实验 2：基本图像处理 1

实验过程中遇到和解决的问题：

（记录实验过程中遇到的问题，以及解决过程和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）

问题 1：MSVC 下测时必须要在 Release 模式并开启编译优化，否则耗时对比无意义。在 Debug 或未开启优化时运行程序，实验 1（split/merge）与实验 2（指针直接遍历）的耗时对比不能反映真实性能差异：未优化时指针版可能因缺少内联、循环优化等反而更慢，且绝对时间会偏大。

解决 1：使用 MSVC 时在 VisualStudio 或 BuildTools 中切换到 Release 配置，并确保启用优化（如/O2）。若使用 task.json 调用 cl.exe，在 args 中增加/O2 和/DNDEBUG，并且在选择 OpenCV 的链接库的时候要选非 Debug 版本：

```

"args": [
    "/nologo",
    "/EHsc",
    "/std:c++17",
    "/O2",
    "/DNDEBUG",
    "/MD",
    "/Zi",
    "${file}",
    "/Fe:${fileDirname}\\${fileBasenameNoExtension}.exe",
    "/I",
    "F:\\library",
    "/I",
    "D:\\OpenCV\\opencv\\build\\include",
    "/link",
    "/LIBPATH:\"D:\\OpenCV\\opencv\\build\\x64\\vc16\\lib\"",
    "opencv_world480.lib",
],

```

这样测得的“实验 1 总时间”与“实验 2 总时间”才具有可比性，指针版一般会明显更快。

问题 2：背景图经 openMat 统一为 4 通道，指针版按 3 通道步进导致崩溃

实验里通过 openMat() 统一把非 4 通道的图像补成 4 通道（与实验 1 一致），因此读入的 back.jpg 实际是 4 通道 BGRA。而实验 2 的指针实现最初假定背景为 3 通道，对每像素做了 pBack+=3。这样在 4 通道数据上指针步进错位，读到的“下一像素”实际是当前像素的 alpha

与下一像素的 B、G，造成内存访问错乱，可能触发栈缓冲区溢出。Windows 下表现为程序在输出实验 1 的测时后崩溃，退出码 -1073740791（即 0xC0000409，STATUS_STACK_BUFFER_OVERRUN）。

解决 2：不再假定背景为 3 通道，根据 `resize` 后的背景图实际通道数步进，只取前 3 个通道参与合成，与实验 1 中“对背景只取 BGR 三通道”的行为一致。核心修改为：

使用 `const int backChannels=backResized.channels();` 得到 3 或 4；

内层循环中改为 `pBack+=backChannels;`，读取时仍只用 `pBack[0]`、`pBack[1]`、`pBack[2]` 作为 BGR。

这样无论背景是 3 通道还是 4 通道，指针都不会越界或错位，程序可稳定跑完并输出两次测时。

代码如下：

```
// ----- 实验 2 优化:指针直接遍历, 图像合成公式( $\alpha$  非二值)
-----
Mat alphaBlend_Exp2_Ptr(Mat& front, Mat& back) {
    CV_Assert(front.channels() == 4);
    Mat backResized;
    resize(back, backResized, front.size());
    // 背景可能是 3 通道(BGR) 或 4 通道(BGRA), 只取前 3 通道参与合成
    const int backChannels = backResized.channels();
    CV_Assert(backChannels == 3 || backChannels == 4);
    const int rows = front.rows;
    const int cols = front.cols;
    Mat blended(rows, cols, CV_8UC3);
    for (int y = 0; y < rows; ++y) {
        const uchar* pFront = front.ptr<uchar>(y); // BGRA, 步
        // 长 4
        const uchar* pBack = backResized.ptr<uchar>(y); // BGR 或
        // BGRA, 步长 backChannels
        uchar* pOut = blended.ptr<uchar>(y); // BGR, 步长
        // 3
        for (int x = 0; x < cols; ++x) {
            // 图像合成公式:  $C_{out} = \alpha * C_{fg} + (1-\alpha) * C_{bg}$ ,  $\alpha$  为归一化透
            // 明度 [0,1], 非二值
            const float a = pFront[3] / 255.0f;
            pOut[0] = cv::saturate_cast<uchar>(a * pFront[0] + (1.0f
            - a) * pBack[0]);
            pOut[1] = cv::saturate_cast<uchar>(a * pFront[1] + (1.0f
            - a) * pBack[1]);
            pOut[2] = cv::saturate_cast<uchar>(a * pFront[2] + (1.0f
            - a) * pBack[2]);
            pFront += 4;
            pBack += backChannels; // 按实际通道数步进, 避免越界
            pOut += 3;
        }
    }
}
```

```
}  
}  
return blended;  
}
```

实验结果分析：

在修复问题 2 后，程序能稳定完成 100 次重复的 alpha 混合，并输出实验 1（split/merge）与实验 2（指针直接遍历）的总时间与平均时间，以及速度比。合成结果保存为`result\_exp1.png`与`result\_exp2.png`，两者在数值上一致（同一公式、同一输入）。

最终输出结果如下：

```
01 PS F:\Homework\计算机视觉作业\实验 2>. \实验 2-图像基本操作-优化.exe  
02 实验 2:Alpha 混合优化与耗时对比  
03 libpng warning: iCCP: known incorrect sRGB profile  
04 ./back.jpg 不是 4 通道图像, 自动补全 alpha 通道  
05 front_img: 924x615, channels=4  
06 back_img: 4000x2250, channels=4  
07 重复次数: 100 (取平均时间)  
08  
09 [实验 1 split/merge] 总时间: 1539.13 ms, 平均: 15.3913 ms/次  
10 [实验 2 指针遍历] 总时间: 235.537 ms, 平均: 2.35537 ms/次  
11  
12 速度比 (实验 1/实验 2): 6.53459 (实验 2 约为实验 1 的 6.53459 倍)  
13 Image saved successfully to result_exp1.png  
14 Image saved successfully to result_exp2.png
```

在 Release 编译优化（MSVC`/O2`）下，实验 2 的指针直接遍历通常明显快于实验 1 的 split/merge 矩阵运算，因为减少了中间 Mat 的分配与多次通道拆分、合并和类型转换。具体加速比与分辨率、重复次数有关，可由控制台打印的“速度比”直接读出。

实验 2 在保留实验 1 图像合成公式与 alpha 连续值处理的前提下，通过指针按行、按像素遍历整张图完成混合，并与实验 1 实现做了运行时间对比；遇到的问题主要集中在 MSVC 下的编译/优化配置、以及背景图通道数与指针步进一致性上，解决后即可得到正确且可比的测时结果。